Sri Sivasubramaniya Nadar College of Engineering, Chennai

Degree & Branch	5 years Integrated M.Tech CSE	**Semester**	V
Subject Code & Name	ICS1512 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Odd Semester)	**Batch**	2023–2028
Name	MadhuVishahan S	**Reg No**	3122237001023

Experiment 5:Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning

Aim:

To implement and compare the performance of:

- Model A: Single-Layer Perceptron Learning Algorithm (PLA).
- Model B: Multilayer Perceptron (MLP) with hidden layers and nonlinear activations.

Students must select and justify hyperparameters such as activation functions, cost functions, optimizers, learning rates, number of hidden layers, and batch sizes through systematic tuning.

Libraries used:

- Numpy
- Pandas
- Matplot Lib
- Scikit-Learn
- Seaborn
- PyTorch

Python Code Implementation

```
    import numpy as np
import pandas as pd
import cv2
import os
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# ------------------------------
# Step 1: Load CSV + Images
# ------------------------------
# img_folder = "Img" # This is not needed as the image path in the CSV is already relative

df = pd.read_csv(r'/content/drive/MyDrive/ML_Experiment5/english.csv')

# Map labels to numeric values (0,1,2,...,61)
label_to_id = {label: idx for idx, label in enumerate(sorted(df['label'].unique()))}
id_to_label = {v: k for k, v in label_to_id.items()}
df['label'] = df['label'].map(label_to_id)

X, y = [], []

for idx, row in df.iterrows():
    # Construct the full path to the image by joining the base directory with the image path f
    img_path = os.path.join('/content/drive/MyDrive/ML_Experiment5/', row['image'])
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE) # grayscale

    if img is not None: # Check if the image was loaded successfully
        img = cv2.resize(img, (28,28))                    # resize
        img = img.flatten() / 255.0                       # flatten + normalize
        X.append(img)
        y.append(row['label'])
    else:
        print(f"Warning: Could not load image from path: {img_path}")


X = np.array(X)
y = np.array(y)

# ------------------------------
# Step 2: Train/Test Split
# ------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ------------------------------
```

```
# Step 3: One-vs-Rest PLA Functions
# ------------------------------
def step_function(z):
    return np.where(z >= 0, 1, 0)

def train_pla_ovr(X, y, num_classes, lr=0.01, epochs=10):
    weights = np.zeros((num_classes, X.shape[1]))
    biases = np.zeros(num_classes)

    for c in range(num_classes):
        y_binary = np.where(y == c, 1, 0) # binary labels for class c
        for _ in range(epochs):
            for xi, target in zip(X, y_binary):
                z = np.dot(xi, weights[c]) + biases[c]
                y_pred = step_function(z)
                update = lr * (target - y_pred)
                weights[c] += update * xi
                biases[c] += update
    return weights, biases

def predict_pla_ovr(X, weights, biases):
    scores = np.dot(X, weights.T) + biases
    return np.argmax(scores, axis=1)

# ------------------------------
# Step 4: Hyperparameter Tuning
# ------------------------------
learning_rates = [0.001, 0.01, 0.1]
epoch_list = [5, 10, 20]

best_acc = 0
best_params = None
best_model = None

for lr in learning_rates:
    for epochs in epoch_list:
        print(f"Training with lr={lr}, epochs={epochs} ...")
        weights, biases = train_pla_ovr(X_train, y_train, len(label_to_id), lr=lr, epochs=epoc
        y_pred = predict_pla_ovr(X_test, weights, biases)
        acc = accuracy_score(y_test, y_pred)
        print(f"Accuracy: {acc:.4f}")

        if acc > best_acc:
            best_acc = acc
            best_params = (lr, epochs)
            best_model = (weights, biases)

print("\n Best Hyperparameters:")
```

```
print("Learning Rate:", best_params[0])
print("Epochs:", best_params[1])
print("Best Accuracy:", best_acc)

# ------------------------------
# Step 5: Final Evaluation
# ------------------------------
final_weights, final_biases = best_model
y_pred_final = predict_pla_ovr(X_test, final_weights, final_biases)

print("\nClassification Report:\n", classification_report(y_test, y_pred_final, target_names=[
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_final))
from sklearn.metrics import accuracy_score, recall_score, f1_score
acc = accuracy_score(y_test, y_pred_final)

# Recall (macro average = equally weights all classes)
recall = recall_score(y_test, y_pred_final, average='macro')

# F1-score (macro average)
f1 = f1_score(y_test, y_pred_final, average='macro')

print(f"\nFinal Metrics:")
print(f"Accuracy : {acc:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")
from sklearn.metrics import roc_curve, auc, RocCurveDisplay
from sklearn.preprocessing import label_binarize
import matplotlib.pyplot as plt

# ------------------------------
# Step 1: Binarize the labels for ROC
# ------------------------------
num_classes = len(label_to_id)
y_test_bin = label_binarize(y_test, classes=range(num_classes))
y_pred_scores = np.dot(X_test, final_weights.T) + final_biases    # use scores before step funct

# ------------------------------
# Step 2: Plot ROC curves for each class
# ------------------------------
plt.figure(figsize=(12, 8))
for i in range(num_classes):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_pred_scores[:, i])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f"{id_to_label[i]} (AUC = {roc_auc:.2f})")

plt.plot([0, 1], [0, 1], "k--")     # diagonal line
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
```

```
plt.title("Multi-class ROC curves (One-vs-Rest)")
plt.legend(loc="lower right", fontsize=8)
plt.show()
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Compute confusion matrix
cm = confusion_matrix(y_test, y_pred_final)

# Set figure size based on number of classes
plt.figure(figsize=(20, 16))

# Create a heatmap
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=[id_to_label[i] for i in range(len(label_to_id))],
            yticklabels=[id_to_label[i] for i in range(len(label_to_id))],
            cbar=True)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix (Improved Visualization)")
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# ------------------------------
# Step 3 (updated): PLA with error history
# ------------------------------
def train_pla_ovr(X, y, num_classes, lr=0.01, epochs=10):
    weights = np.zeros((num_classes, X.shape[1]))
    biases = np.zeros(num_classes)

    # store overall error per epoch
    error_history = []

    for epoch in range(epochs):
        total_errors = 0

        for c in range(num_classes):
            y_binary = np.where(y == c, 1, 0)
            for xi, target in zip(X, y_binary):
                z = np.dot(xi, weights[c]) + biases[c]
                y_pred = 1 if z >= 0 else 0
                update = lr * (target - y_pred)
                if update != 0:
                    total_errors += 1
```

```
                weights[c] += update * xi
                biases[c] += update

        # error rate = total errors / (samples × classes)
        error_rate = total_errors / (len(y) * num_classes)
        error_history.append(error_rate)

    return weights, biases, error_history


# ------------------------------
# After training loop
# ------------------------------
weights, biases, error_history = train_pla_ovr(X_train, y_train, len(label_to_id), lr=0.01, ep

# ------------------------------
# Plot Training Error vs Epochs
# ------------------------------
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.plot(range(1, len(error_history)+1), error_history, marker='o')
plt.xlabel("Epochs")
plt.ylabel("Training Error Rate")
plt.title("PLA Training Error vs Epochs")
plt.grid(True)
plt.show()

import os
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
import pandas as pd
import torch
import random

# --------------------------------
# Config
# --------------------------------
IMG_SIZE = (28, 28)
RANDOM_SEED = 42
NUM_CLASSES = 62
BASE_DIR = "/content/drive/MyDrive/ML_Experiment5"      # <-- your project folder

np.random.seed(RANDOM_SEED)
torch.manual_seed(RANDOM_SEED)
```

```
random.seed(RANDOM_SEED)

# ------------------------------
# Load CSV
# ------------------------------
csv_path = os.path.join(BASE_DIR, "english.csv")
df = pd.read_csv(csv_path)
print("CSV columns:", df.columns)
print(df.head())

# ------------------------------
# Image Loader
# ------------------------------
def load_images_from_csv(csv_df, base_dir=BASE_DIR, img_size=(28,28)):
    X, y = [], []
    for _, row in csv_df.iterrows():
        rel_path = row["image"]                          # e.g., "Img/img001-001.png"
        full_path = os.path.join(base_dir, rel_path) # -> /content/.../ML_Experiment5/Img/img0
        if os.path.exists(full_path):
            img = Image.open(full_path).convert("L").resize(img_size)
            arr = np.array(img, dtype=np.float32) / 255.0
            X.append(arr.flatten())
            y.append(row["label"])
        else:
            print("Missing file:", full_path)
    return np.array(X), np.array(y)

# ------------------------------
# Load dataset
# ------------------------------
X, y_raw = load_images_from_csv(df, base_dir=BASE_DIR, img_size=IMG_SIZE)
print("Dataset:", X.shape, "Unique labels:", len(set(y_raw)))

# ------------------------------
# Encode labels & split
# ------------------------------
le = LabelEncoder()
y = le.fit_transform(y_raw)

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=RANDOM_SEED
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=RANDOM_SEED
)

print("Train:", X_train.shape, "Val:", X_val.shape, "Test:", X_test.shape)
```

```
# ------------------------------
# Normalize features
# ------------------------------
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_val_s = scaler.transform(X_val)
X_test_s = scaler.transform(X_test)
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_layers, num_classes, activation='relu', dropout=0.0):
        super().__init__()
        layers, prev = [], input_dim
        act_map = {'relu':nn.ReLU, 'tanh':nn.Tanh, 'sigmoid':nn.Sigmoid}
        for h in hidden_layers:
            layers += [nn.Linear(prev, h), act_map[activation]()]
            if dropout>0: layers.append(nn.Dropout(dropout))
            prev = h
        layers.append(nn.Linear(prev, num_classes))
        self.net = nn.Sequential(*layers)
    def forward(self, x): return self.net(x)

# Training Utilities
def make_loader(X, y, batch=64, shuffle=True):
    ds = TensorDataset(torch.tensor(X, dtype=torch.float32), torch.tensor(y, dtype=torch.long))
    return DataLoader(ds, batch_size=batch, shuffle=shuffle)

def train_epoch(model, loader, opt, device):
    model.train(); total=0
    for xb,yb in loader:
        xb,yb = xb.to(device), yb.to(device)
        out = model(xb)
        loss = F.cross_entropy(out, yb)
        opt.zero_grad(); loss.backward(); opt.step()
        total += loss.item()*xb.size(0)
    return total/len(loader.dataset)

def eval_model(model, loader, device):
    model.eval(); preds, labs = [], []
    total=0
    with torch.no_grad():
        for xb,yb in loader:
            xb,yb = xb.to(device), yb.to(device)
            out = model(xb)
            loss = F.cross_entropy(out, yb)
            total += loss.item()*xb.size(0)
            preds.append(out.argmax(1).cpu().numpy())
            labs.append(yb.cpu().numpy())
    return total/len(loader.dataset), np.concatenate(preds), np.concatenate(labs)
```

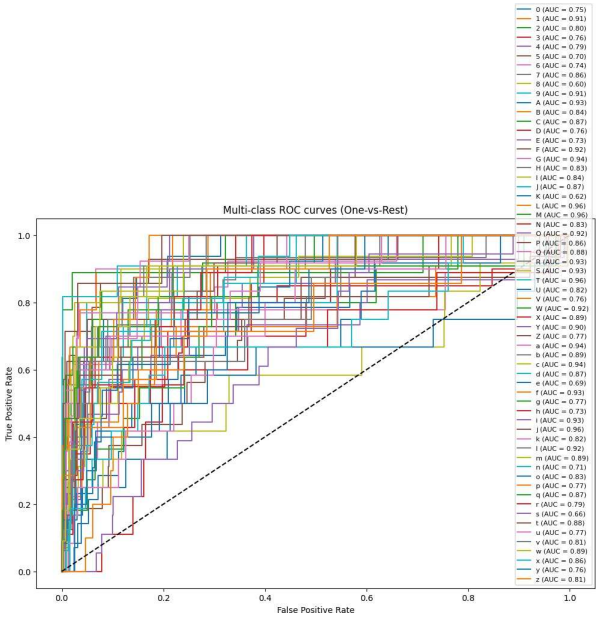
```
# Hyperparameter grid
hyperparams = {
    'hidden_layers': [[128], [256], [256,128], [512,256]],
    'activation': ['relu', 'tanh', 'sigmoid'],
    'dropout': [0.0, 0.2, 0.5],
    'lr': [1e-2, 1e-3, 1e-4],
    'batch_size': [64, 128, 256],
    'optimizer': ['adam', 'sgd', 'rmsprop']
}

# Train MLP
device = "cuda" if torch.cuda.is_available() else "cpu"

best_val_acc = 0
best_config = None
best_model_state = None

# Iterate over all hyperparameter combinations
for hl, act, do, lr, bs, opt_name in product(
    hyperparams['hidden_layers'],
    hyperparams['activation'],
    hyperparams['dropout'],
    hyperparams['lr'],
    hyperparams['batch_size'],
    hyperparams['optimizer']
):
    # Prepare loaders with current batch size
    train_loader = make_loader(X_train_s, y_train, batch=bs)
    val_loader = make_loader(X_val_s, y_val, batch=bs, shuffle=False)

    # Initialize model and optimizer
    model = MLP(X_train_s.shape[1], hl, NUM_CLASSES, activation=act, dropout=do).to(device)
    if opt_name == 'adam':
        opt = optim.Adam(model.parameters(), lr=lr)
    elif opt_name == 'sgd':
        opt = optim.SGD(model.parameters(), lr=lr, momentum=0.9)     # momentum is standard for S
    elif opt_name == 'rmsprop':
        opt = optim.RMSprop(model.parameters(), lr=lr)

    # Train for a few epochs (e.g., 5 for tuning speed)
    for epoch in range(5):
        train_epoch(model, train_loader, opt, device)

    # Evaluate on validation set
    _, y_val_pred, y_val_true = eval_model(model, val_loader, device)
    val_acc = (y_val_pred == y_val_true).mean()

    # Save best
```

```
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_config = {'hidden_layers': hl, 'activation': act, 'dropout': do, 'lr': lr, 'batch
        best_model_state = model.state_dict()

print("Best hyperparameters:", best_config, "Validation Accuracy:", best_val_acc)

train_loader = make_loader(X_train_s, y_train, batch=best_config['batch_size'])
val_loader = make_loader(X_val_s, y_val, batch=best_config['batch_size'], shuffle=False)
test_loader = make_loader(X_test_s, y_test, batch=best_config['batch_size'], shuffle=False)

model = MLP(X_train_s.shape[1], best_config['hidden_layers'], NUM_CLASSES,
            activation=best_config['activation'],        dropout=best_config['dropout']).to(device)
model.load_state_dict(best_model_state)     # Optionally start from tuned weights

if best_config['optimizer'] == 'adam':
    opt = optim.Adam(model.parameters(), lr=best_config['lr'])
elif best_config['optimizer'] == 'sgd':
    opt = optim.SGD(model.parameters(), lr=best_config['lr'], momentum=0.9)
elif best_config['optimizer'] == 'rmsprop':
    opt = optim.RMSprop(model.parameters(), lr=best_config['lr'])

history = {'train_loss':[], 'val_loss':[], 'val_acc':[]}
for epoch in range(20):
    tl = train_epoch(model, train_loader, opt, device)
    vl, vp, vlabs = eval_model(model, val_loader, device)
    acc = (vp==vlabs).mean()
    history['train_loss'].append(tl); history['val_loss'].append(vl); history['val_acc'].appen
    print(f"Epoch {epoch+1}: train={tl:.3f}, val={vl:.3f}, acc={acc:.3f}")
# Evaluation (MLP vs PLA)
# Test set
_, y_pred_mlp, _ = eval_model(model, test_loader, device)
print("MLP Classification Report:\n", classification_report(y_test, y_pred_mlp))

# Confusion Matrix Example (MLP)
cm = confusion_matrix(y_test, y_pred_mlp)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")
plt.title("MLP Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()



# ROC (micro-average)
def predict_proba(model, loader):
    model.eval(); out=[]
```

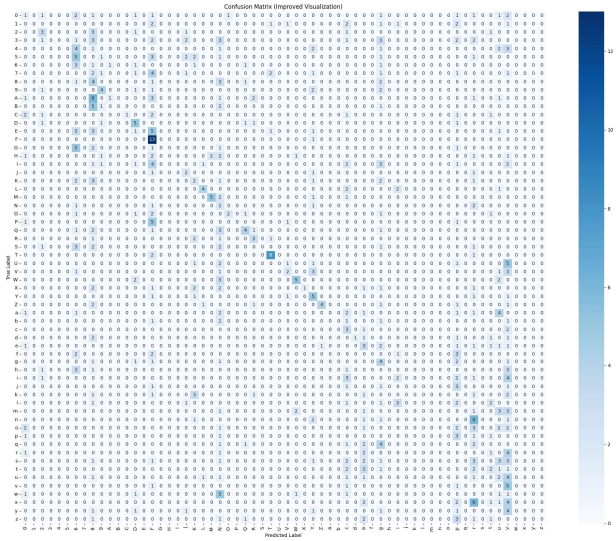
```
    with torch.no_grad():
        for xb,_ in loader:
            logits = model(xb.to(device))
            out.append(F.softmax(logits,1).cpu().numpy())
    return np.vstack(out)

probs = predict_proba(model, test_loader)
y_onehot = np.eye(NUM_CLASSES)[y_test]
fpr,tpr,_ = roc_curve(y_onehot.ravel(), probs.ravel())
plt.plot(fpr,tpr);      plt.plot([0,1],[0,1],'k--')
plt.title("Micro-average ROC"); plt.show()

# Plot Training Curves
plt.plot(history['train_loss'],  label="train_loss")
plt.plot(history['val_loss'], label="val_loss")
plt.plot(history['val_acc'], label="val_acc")
plt.legend(); plt.title("Training Curves"); plt.show()
```

Plots and Visualizations

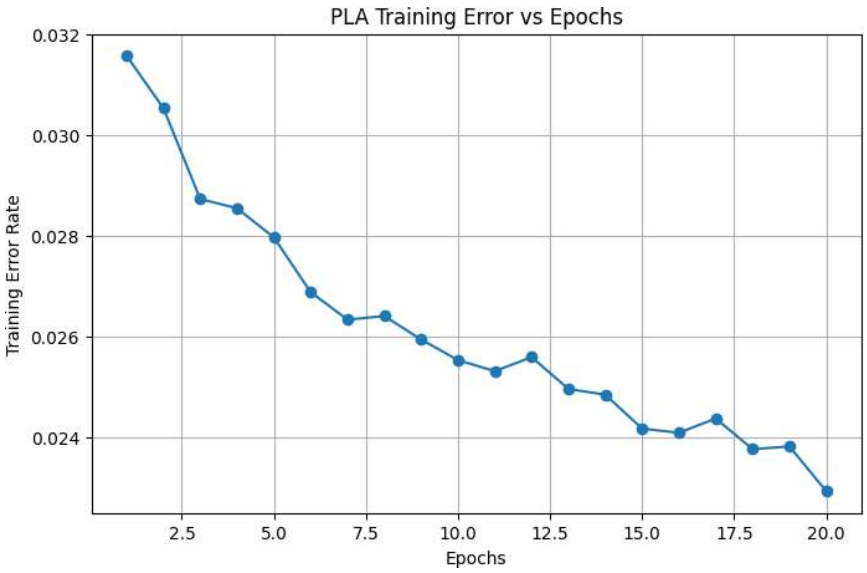
PLA



(a) ROC Curve



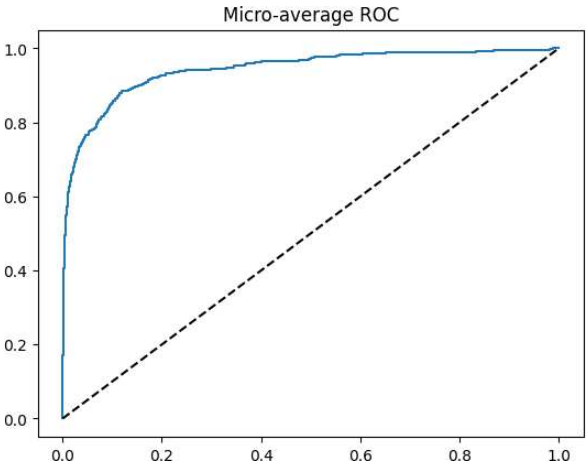
(b) Confusion Matrix



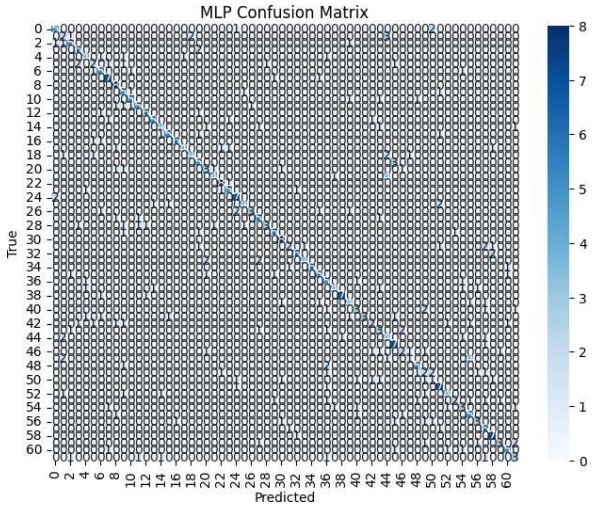
(c) Training Error vs Epochs

Figure 1: Performance of PLA Classifier: (a) ROC Curve, (b) Confusion Matrix, and (c) Training Error vs Epochs

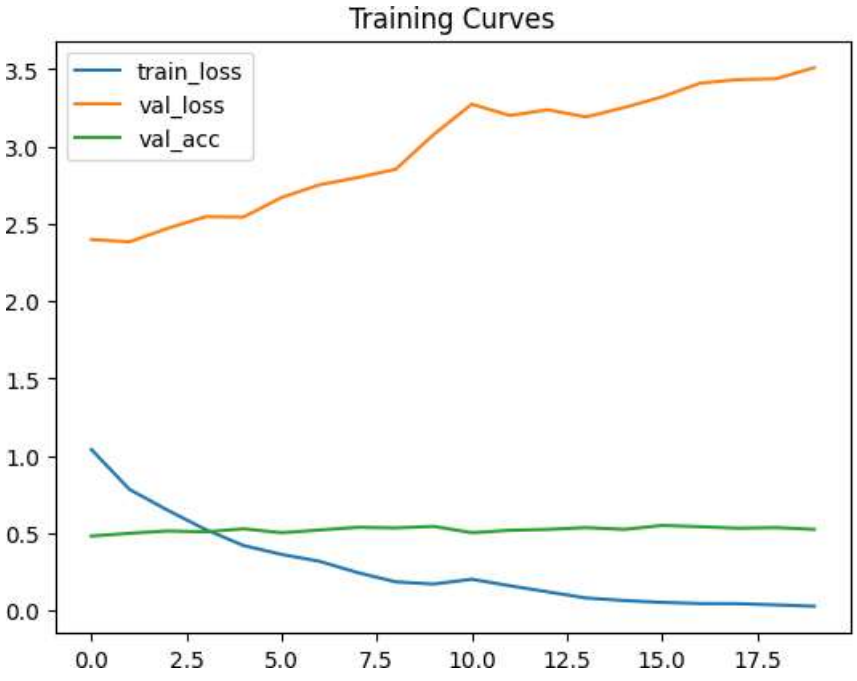
MLP



(a) ROC Curve



(b) Confusion Matrix



(c) Training Error vs Epochs

Figure 2: Performance of MLP Classifier: (a) ROC Curve, (b) Confusion Matrix, and (c) Training Error vs Epochs

Results

PLA

Hyperparameter Table

Table 1: PLA- Hyperparameter Trials

Learning Rate	Epochs	Accuracy
0.01	20	0.1598240469208211

Classification Report

- Accuracy : 0.1598
- Recall : 0.1748
- F1 Score : 0.1355

MLP

Hyperparameter Table

Table 2: MLP - Hyperparameters Trials

Hyperparameter	Value
Hidden Layers	[512, 256]
Activation	relu
Dropout	0.0
Learning Rate	0.001
Batch Size	64
Optimizer	adam
Epochs	20

Classification Report

- Accuracy: 0.54
- Precision: 0.56
- Recall: 0.54
- F1-score: 0.53

Observations

- The Perceptron Learning Algorithm (PLA) performed poorly on the dataset, achieving only about 16% accuracy. This shows that a simple linear classifier is not sufficient for high-dimensional, non-linearly separable data such as handwritten character recognition.
- PLA exhibited high training error even after several epochs, which indicates its inability to capture complex patterns in the data.
- The confusion matrix for PLA revealed a large number of misclassifications across multiple classes, further supporting the limitation of linear models for multi-class classification.
- In contrast, the Multilayer Perceptron (MLP) achieved significantly higher accuracy (around 54%), demonstrating its ability to learn non-linear decision boundaries using hidden layers and activation functions.
- Hyperparameter tuning (hidden layers, activation function, optimizer, learning rate, dropout, and batch size) played a crucial role in improving MLP performance. The best configuration was found with hidden layers [512, 256], ReLU activation, Adam optimizer, learning rate = 0.001, and batch size = 64.
- The ROC curve and confusion matrix of MLP indicated better class separation compared to PLA, although performance is still limited by the size and complexity of the dataset.
- Training error curves showed that PLA stabilized quickly without much improvement, while MLP exhibited gradual learning with reduced error and improved validation accuracy across epochs.

Learning Outcomes

- Understood the working principle of the Perceptron Learning Algorithm and its limitation in handling multi-class, non-linearly separable data.
- Gained practical experience in designing and training a Multilayer Perceptron with different hyperparameters.
- Learned the importance of systematic hyperparameter tuning (learning rate, batch size, hidden layers, activation functions, and optimizers) for improving model performance.
- Explored evaluation metrics such as Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC Curves for analyzing classification models.
- Observed how deep learning models like MLP significantly outperform traditional PLA in real-world datasets by capturing non-linear relationships.
- Developed skills in visualizing model performance through error curves, ROC curves, and confusion matrices.
- Understood that although MLP performs better, further improvement may require deeper architectures, more data, or advanced techniques such as CNNs.